

Distributed Systems

Lecture11: Client and Server

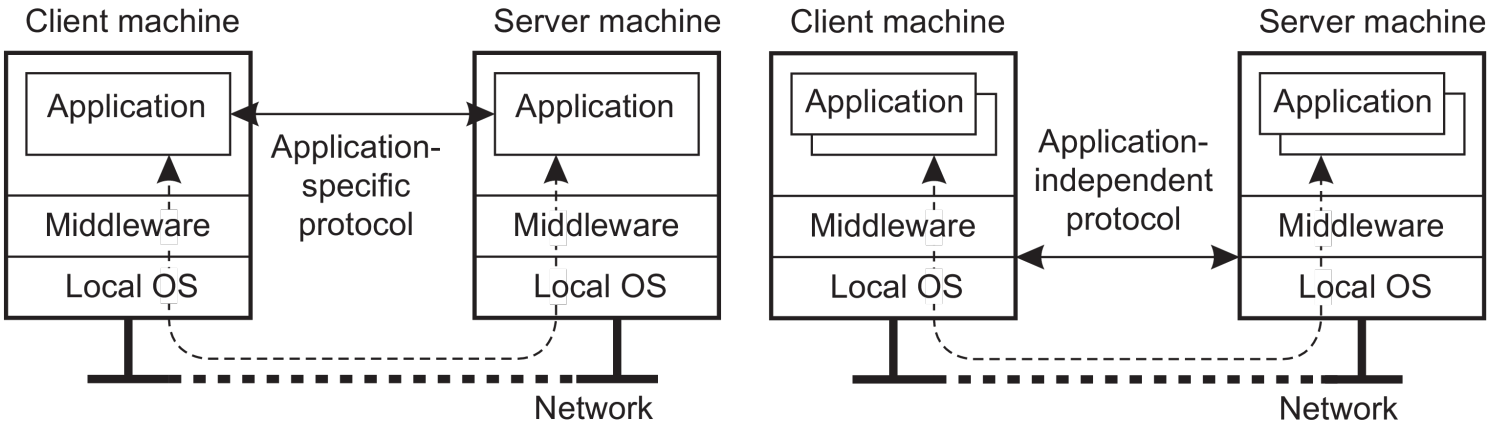
Mohammad Reza Shakournia

shakournia@ut.ac.ir

School of Electrical and Computer Engineering,
University of Tehran,
Tehran, Iran

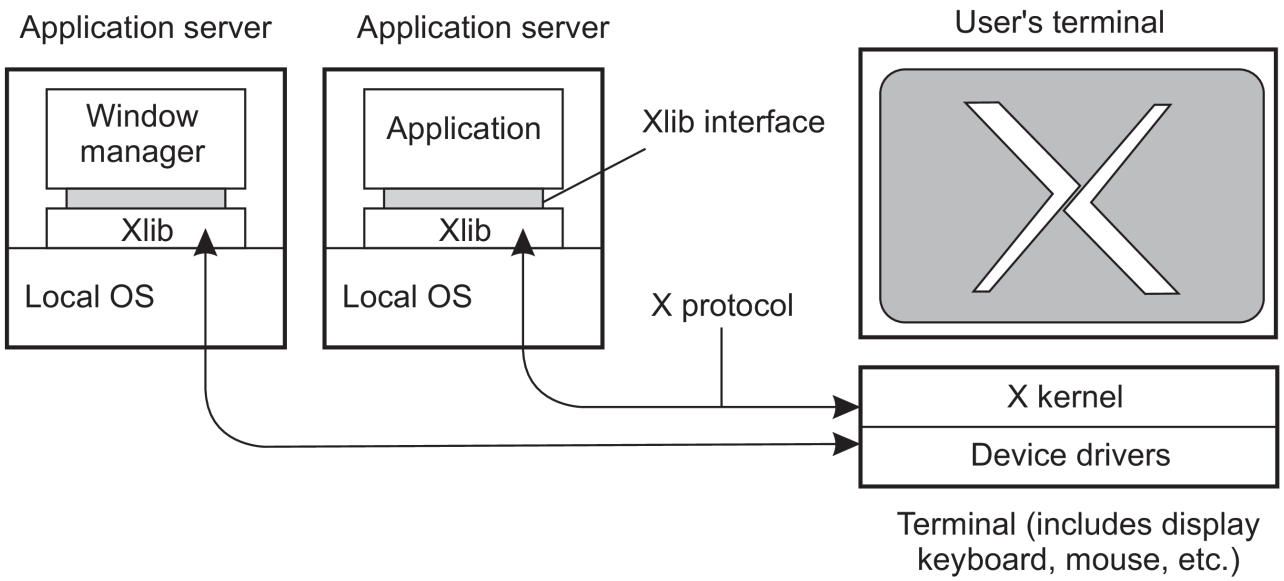
Client-Server interaction

- Distinguish application-level and middleware-level solutions



Example: The X Window System

- Basic organization



- X client and server:
  - The application acts as a client to the X-kernel, the latter running as a server on the client's machine.

Improving X

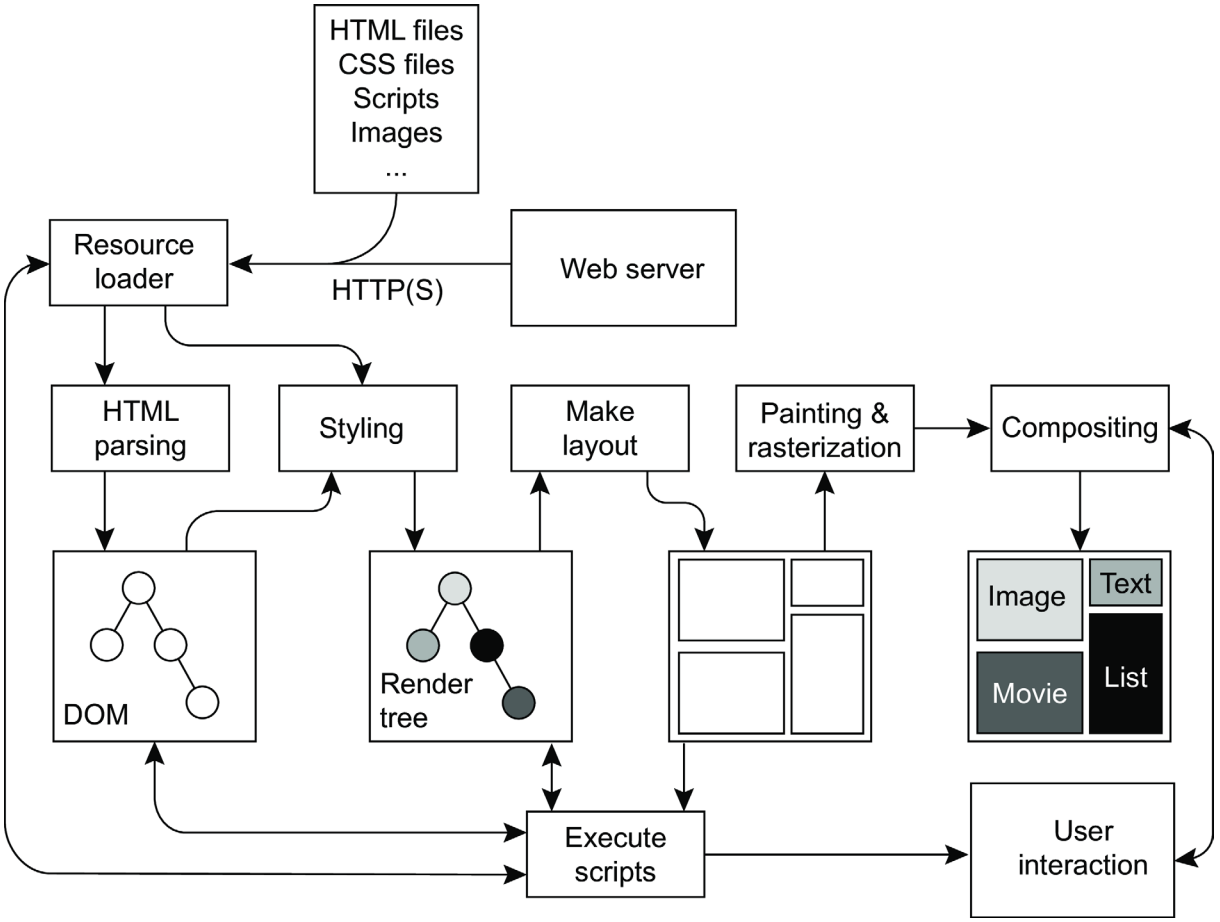
- Practical observations
  - There is often no clear separation between application logic and user-interface commands
  - Applications tend to operate in a tightly synchronous manner with an X kernel
- Alternative approaches
  - Let applications control the display completely, up to the pixel level (e.g., VNC)
  - Provide only a few high-level display operations (dependent on local video drivers), allowing more efficient display operations.

Virtual Desktop Environment

- **Logical development**
- With an increasing number of cloud-based applications, the question is how to use those applications from a user's premise?
  - Issue: develop the ultimate networked user interface
  - Answer: use a Web browser to establish a seamless experience

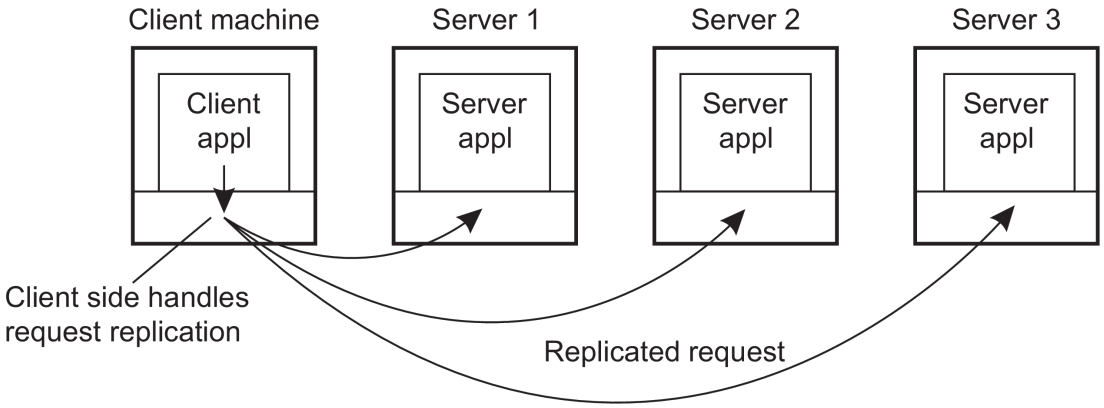


Anatomy of Web Browser



Client-Side Software

- Access transparency: client-side stubs for RPCs
- Location/migration transparency: let client-side software keep track of actual location
- Replication transparency: multiple invocations handled by client stub:



- Failure transparency: can often be placed only at client (we're trying to mask server and communication failures).

Servers: General Organization

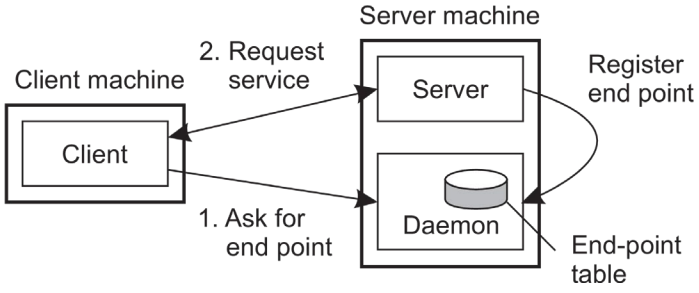
- A process implementing a specific service on behalf of a collection of clients. It waits for an incoming request from a client and subsequently ensures that the request is taken care of, after which it waits for the next incoming request.
- **Two basic types:**
  - Iterative server: Server handles the request before attending a next request.
  - Concurrent server: Uses a dispatcher, which picks up an incoming request that is then passed on to a separate thread/process.
- Concurrent servers are the norm: they can easily handle multiple requests, notably in the presence of blocking operations (to disks or other servers).

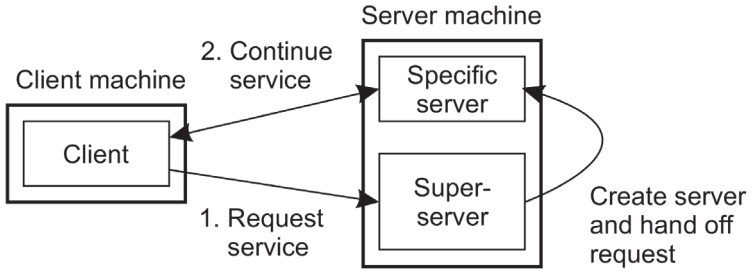
Contacting a Server

- Observation: most services are tied to a specific port

ftp-data	20	File Transfer [Default Data]
ftp	21	File Transfer [Control]
telnet	23	Telnet
Smtp	25	Simple Mail Transfer
www	80	Web (HTTP)

- Dynamically assigning an end point: two approaches





Out-of-Band Communication

- **Issue:** Is it possible to interrupt a server once it has accepted (or is in the process of accepting) a service request?
- **Solution 1: Use a separate port for urgent data**
  - Server has a separate thread/process for urgent messages
  - Urgent message comes in ⇒ associated request is put on hold
  - Note: we require OS supports priority-based scheduling
- **Solution 2: Use facilities of the transport layer**
  - Example: TCP allows for urgent messages in same connection
  - Urgent messages can be caught using OS signaling techniques

Servers and State

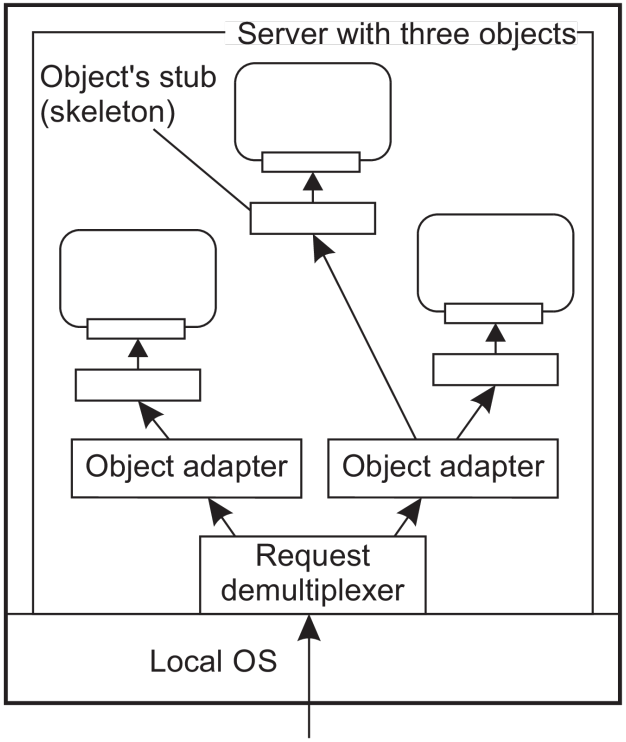
- **Stateless servers**
- Never keep accurate information about the status of a client after having handled a request:
  - Don’t record whether a file has been opened (simply close it again after access)
  - Don’t promise to invalidate a client’s cache
  - Don’t keep track of your clients
- **Consequences**
  - Clients and servers are completely independent
  - State inconsistencies due to client or server crashes are reduced
  - Possible loss of performance because, e.g., a server cannot anticipate client behavior (think **of prefetching file blocks)**

Servers and State

- **Stateful servers**
- Keeps track of the status of its clients:
  - Record that a file has been opened, so that prefetching can be done
  - Knows which data a client has cached, and allows clients to keep local copies of shared data
- The performance of stateful servers can be extremely high, provided clients are allowed to keep local copies. As it turns out, reliability is often not a major problem.

Object Servers

- **Activation policy:** which actions to take when an invocation request comes in:
  - Where are code and data of the object?
  - Which threading model to use?
  - Keep modified state of object, if any?
- **Object adapter:** implements a specific activation policy



Example: Ice Runtime System – A Server

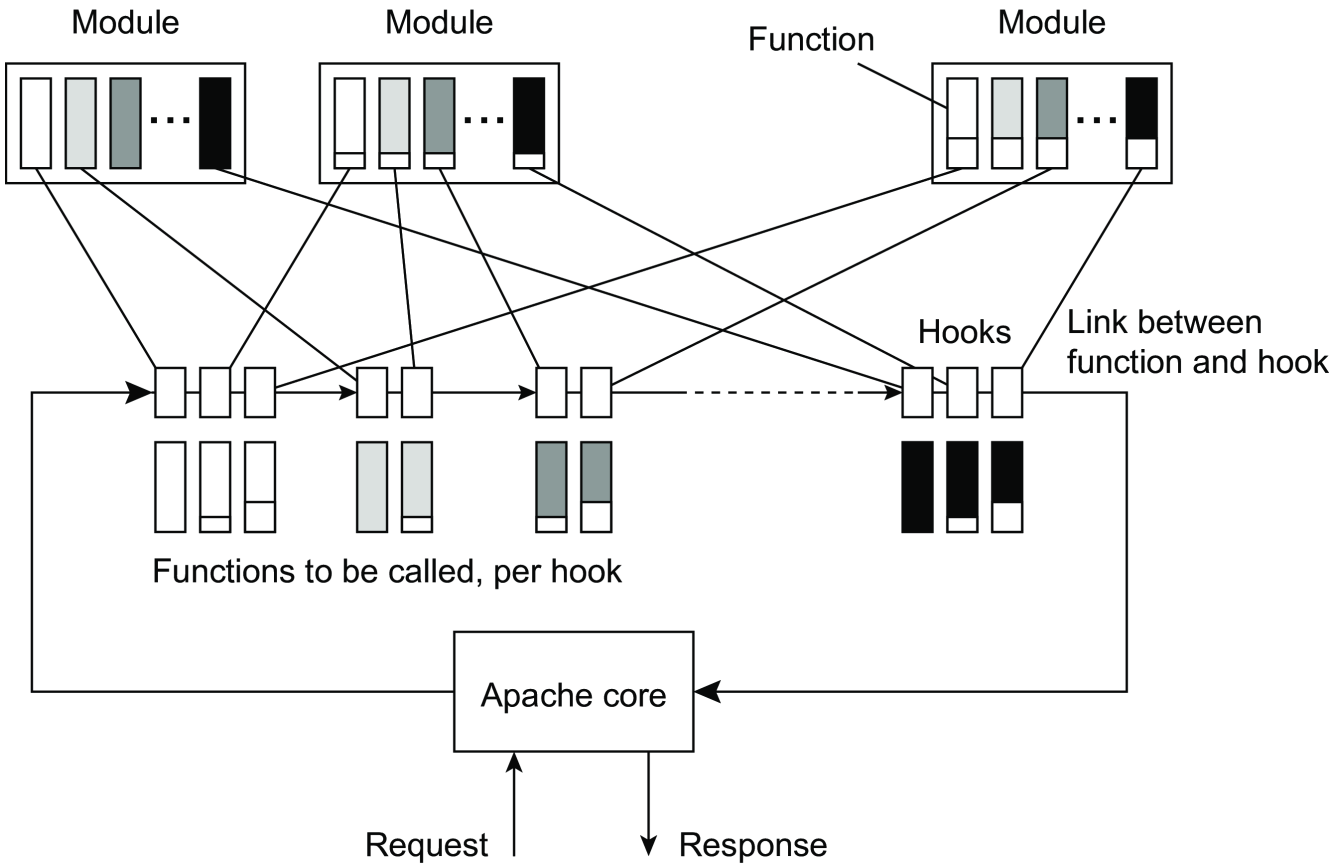
```
1  import sys, Ice
2  import Demo
3
4  class PrinterI(Demo.Printer):
5      def __init__(self, t):
6          self.t = t
7
8      def printString(self, s, current=None):
9          print(self.t, s)
10
11 communicator = Ice.initialize(sys.argv)
12
13 adapter = communicator.createObjectAdapterWithEndpoints("SimpleAdapter", "default -p 1100
14 object1 = PrinterI("Object1 says:")
15 object2 = PrinterI("Object2 says:")
16 adapter.add(object1, communicator.stringToIdentity("SimplePrinter1"))
17 adapter.add(object2, communicator.stringToIdentity("SimplePrinter2"))
18 adapter.activate()
19
20 communicator.waitForShutdown()
```

Example: Ice Runtime System – A Client

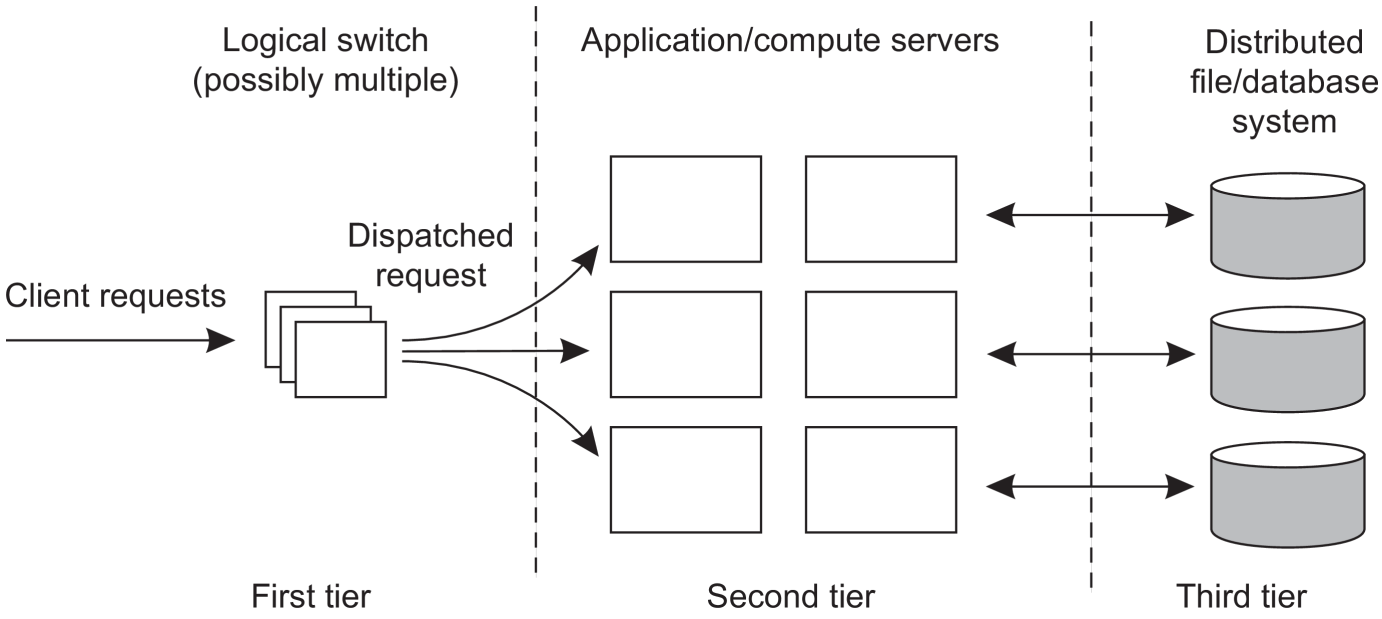
```
1  import sys, Ice
2  import Demo
3
4  communicator = Ice.initialize(sys.argv)
5
6  base1 = communicator.stringToProxy("SimplePrinter1:default -p 11000")
7  base2 = communicator.stringToProxy("SimplePrinter2:default -p 11000")
8  printer1 = Demo.PrinterPrx.checkedCast(base1)
9  printer2 = Demo.PrinterPrx.checkedCast(base2)
10 if (not printer1) or (not printer2):
11     raise RuntimeError("Invalid proxy")
12
13 printer1.printString("Hello World from printer1!")
14 printer2.printString("Hello World from printer2!")
15
16 communicator.waitForShutdown()
```

Object1 says: Hello World from printer1!
Object2 says: Hello World from printer2!

Example: The Apache Web Server



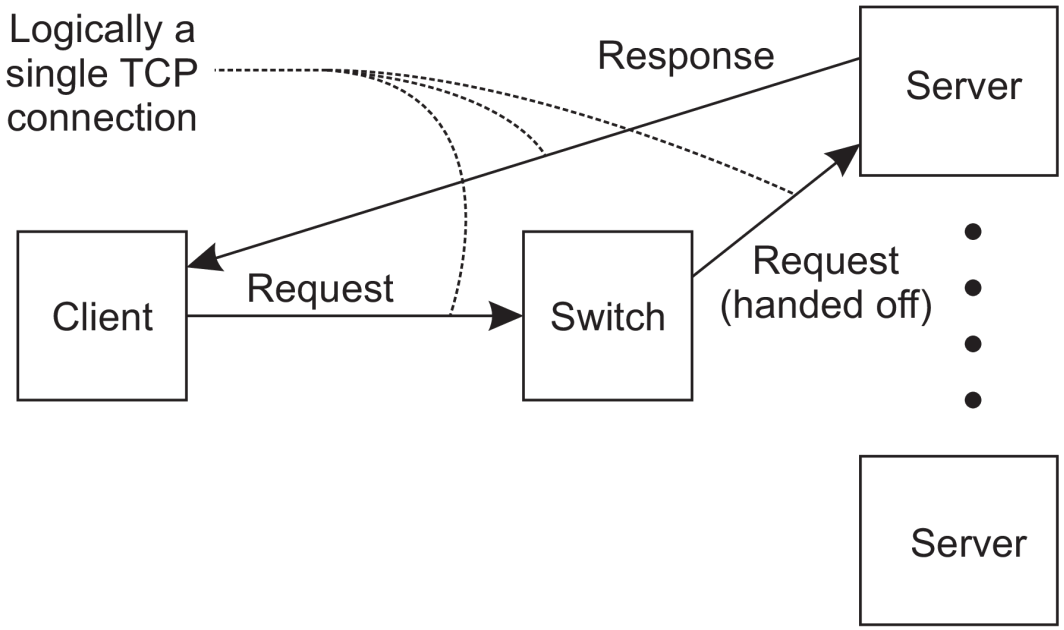
Three Different Tiers



- **Crucial element:**
  - The first tier is generally responsible for passing requests to an appropriate server: request dispatching

Request Handling

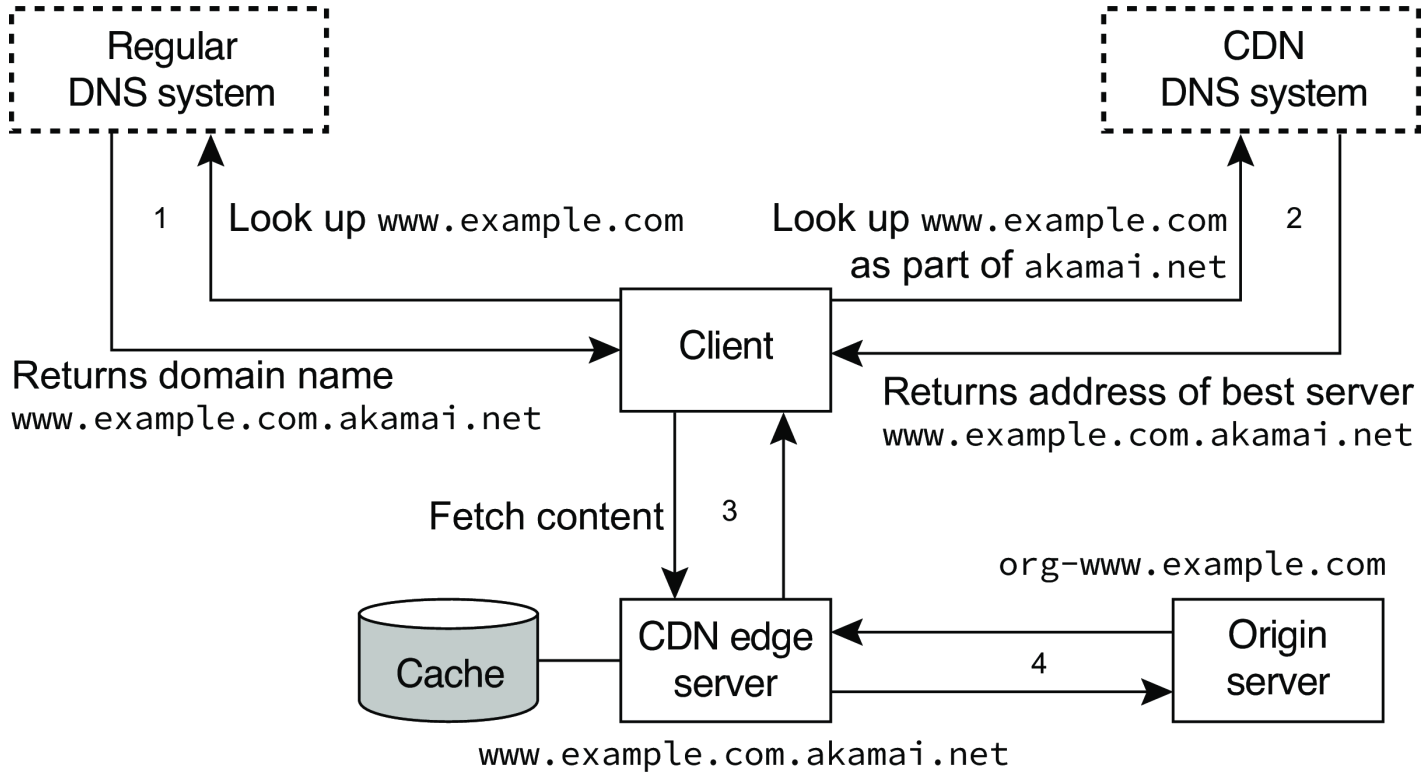
- Having the first tier handle all communication from/to the cluster may lead to a bottleneck.
- **A solution: TCP handoff**



When Servers are Spread Across the Internet

- Spreading servers across the Internet may introduce administrative problems. These can be largely circumvented by using data centers from a single cloud provider.
- **Request dispatching: if locality is important!**
- Common approach: use DNS:
    1. Client looks up specific service through DNS - client's IP address is part of request
    2. DNS server keeps track of replica servers for the requested service, and returns address of most local server.
- **Client Transparency:** To keep client unaware of distribution, let DNS resolver act on behalf of client. Problem is that the resolver may actually be far from local to the actual client.

A Simplified Version of The Akamai CDN



- The cache is often sophisticated enough to hold more than just passive data. **Much of the application code** of the origin server can be moved to the cache as well.

Wrap-Up + What's Next

In the next lecture, we will discuss the situations in which passing programs, sometimes even while they are being executed, simplifies the design of a distributed system.

- Reasons for Migrating codes,
- Models for code migrations,
- Migration in heterogenous systems.